

RAÍZ: de red de pagos turísticos a protocolo de ahorro comunitario sobre Stellar

Diseño técnico para la independencia de yield y la capa de enjambre

Protocolo Raíz

Borrador técnico v1.0 — 31 de julio de 2026

raizapp.xyz

Resumen. RAÍZ es una red de pagos turísticos sobre Stellar que desvía un porcentaje configurable de cada pago (*Tip Barrio*, 2%) a un fondo comunitario custodiado por contratos Soroban y gobernado por residentes mediante tokens *soulbound*. El sistema actual —cuatro contratos desplegados en testnet, una aplicación Android nativa y flujos verificados extremo a extremo— delega el rendimiento del fondo en el vault USDC de DeFindex. Este documento presenta el diseño técnico para convertir a RAÍZ en un *protocolo de ahorro comunitario* autónomo, articulado en cuatro vías: (i) la sustitución de DeFindex por integración directa con Blend detrás de una interfaz de adaptación propia (*YieldAdapter*), gobernable por los residentes; (ii) una capa de instrumentos de ahorro que no existe hoy en el ecosistema Stellar —círculos de ahorro rotativo con identidad *soulbound*, bóvedas de metas con bloqueo temporal, ahorro con compromiso y sorteo sin pérdida—; (iii) la capa *Enjambre*, en la que los dispositivos del barrio participan de la infraestructura del protocolo: custodia colectiva del fondo mediante cuentas inteligentes con *passkeys* y, en su etapa final, firmas umbral FROST; atestación vecinal de residencia como red de confianza; transporte de transacciones por malla *store-and-forward*; verificación ligera de finalidad SCP en el móvil; y recompensas tipo DePIN financiadas por la comisión del protocolo; y (iv) una capa de *conocimiento cero* bajo el principio “transparencia colectiva, privacidad individual”: voto secreto tipo Semaphore verificado con las primitivas BN254/Poseidon de los Protocolos 25–26 —la primera votación privada del ecosistema— y, en fase posterior, privacidad de saldos individuales. Se analiza la viabilidad de cada componente contra el estado verificado de la red Stellar a julio de 2026 (Protocol 27), se delimita explícitamente lo que la física de los sistemas distribuidos —y los límites reales del voto ZK— no permiten prometer, y se propone una hoja de ruta por fases compatible con el Stellar Community Fund.

Palabras clave: Stellar, Soroban, ahorro comunitario, ROSCA, firmas umbral, FROST, smart accounts, *passkeys*, DePIN, SCP, zero-knowledge, Semaphore, Groth16.

1 Introducción

El turismo extrae valor de los barrios que lo hacen posible: las plataformas de intermediación, las cadenas y la plusvalía capturan el margen, mientras quienes crean el atractivo —residentes, artesanos, comercio de esquina— no participan de las decisiones de reinversión. RAÍZ ataca este problema con una tesis simple y verificable: *el valor que genera el barrio debe quedarse en el barrio y decidirse en el barrio, on-chain*.

La implementación actual, desarrollada como MVP y desplegada en la testnet de Stellar el 29 de junio de 2026, demuestra la tesis en producción técnica: un turista paga en USDC a un comercio; el contrato `Pool` separa automáticamente el monto del comercio, el *Tip Barrio* (2%) hacia el fondo comunitario y la comisión del protocolo (0,5%); los residentes —acreditados con un token de residencia intransferible— proponen y votan el destino del fondo; y el contrato `Treasury` ejecuta las propuestas aprobadas sin que ninguna llave privada individual controle los fondos. Todos los eventos son auditables públicamente.

El sistema actual tiene, sin embargo, una dependencia estructural: el rendimiento financiero del fondo ocioso se obtiene depositándolo en el vault USDC de DeFindex. Esa dependencia impulsó la elección del token (el USDC de Blend en testnet, único aceptado por el vault), añade una capa de comisión y contraparte, y requiere una API externa para reportar el APY. Este documento responde a una pregunta de diseño concreta: **¿cómo se convierte RAÍZ en un protocolo de ahorro por derecho propio, sin DeFindex, aprovechan-**

do la arquitectura de Stellar, y construyendo tecnología que hoy no existe en ese ecosistema, sin comprometer la misión comunitaria?

1.1 Contribuciones

- C1. Un análisis verificado del estado de la red Stellar y su ecosistema de ahorro a julio de 2026, incluyendo un *gap analysis* de los instrumentos que no existen (§2, §2.4).
- C2. El diseño de *YieldAdapter*: la sustitución quirúrgica de DeFindex por integración directa con Blend detrás de una interfaz propia y gobernable (§4).
- C3. El diseño de la capa de ahorro comunitario: *Cadena de Barrio* (ROSCA *soulbound*), bóvedas de metas, ahorro con compromiso y sorteo sin pérdida (§5).
- C4. La capa *Enjambre*: custodia colectiva (*passkeys* → FROST), atestación vecinal, malla de transporte de transacciones, verificación ligera SCP y recompensas DePIN (§6).
- C5. La capa ZK: voto secreto tipo Semaphore sobre las primitivas BN254/Poseidon de Protocol 25–26 —inédito en Stellar— y privacidad de saldos individuales como fase posterior (§7).
- C6. Un modelo de amenazas honesto que delimita lo técnicamente imposible —validación móvil del consenso global, finalidad offline sin hardware seguro, voto incoercible— frente a lo construible (§8).

Tabla 1: Protocolos Stellar 2025–2026 (verificado jul-2026).

P.	Mainnet	Aporte principal
23	sep-25	Ejecución paralela Soroban (CAP-63); estado vivo en RAM y auto-restore (CAP-66); caché Wasm; eventos unificados; timing SCP votable (CAP-70) [6]
24	oct-25	Estabilidad (fix de <i>state archival</i>) [7]
25	ene-26	“X-Ray”: verificación ZK on-chain (BN254, CAP-74) y hashes Poseidon (CAP-75) [8]
26	may-26	“Yardstick”: TTL v2 (CAP-78); congelamiento por consenso (CAP-77); aritmética 256-bit; mejoras SAC [9]
27	jul-26	“Zipper”: delegación de autenticación nativa para smart accounts (CAP-71); antesala de Protocol 28 [10]

2 Contexto: la red Stellar en 2026

2.1 Consenso y estructura

Stellar alcanza consenso mediante SCP (*Stellar Consensus Protocol*), una instancia de acuerdo bizantino federado (FBA): cada nodo declara su *quorum set* —los nodos en los que confía y el umbral requerido— y el consenso emerge de la intersección transitiva de esas elecciones (*quorum slices*) [1, 2]. SCP prioriza *safety* sobre *liveness*: ante un fallo de quórum la red se detiene antes que bifurcarse. No existe minería ni *staking*; validar no genera recompensas de protocolo. El núcleo de confianza público (*tier-1*) estaba compuesto por siete organizaciones con un plan activo de expansión a trece; el 16 de julio de 2026 se anunció la incorporación de MoneyGram, Figure Markets y Range [3, 4].

Los requisitos de un validador (8 núcleos, 16 GB de RAM —al alza tras CAP-66—, NVMe de 10 000 IOPS, uptime del 99,9 % [5]) son relevantes para §6: delimitan qué puede y qué no puede ser un dispositivo móvil dentro del protocolo.

2.2 La cadena de protocolos 2025–2026

La Tabla 1 resume los *upgrades* verificados. Dos son estructurales para este diseño: **Protocol 23** introdujo la ejecución paralela de contratos y el estado vivo en memoria (lecturas más baratas, restauración automática de entradas archivadas) [6], y **Protocol 27** (“Zipper”, mainnet 8-jul-2026) introdujo la *delegación de autenticación nativa* para cuentas inteligentes (CAP-71): un firmante puede delegar en otros, agrupando firmantes delegados en una única entrada de autorización [10]. Protocol 28 completará la autenticación basada en contratos. Es exactamente el primitivo que la custodia de enjambre necesita (§6.1); RAÍZ, que ya opera smart accounts con passkeys, llega a esta ola en el momento en que la SDF la está construyendo.

2.3 Ecosistema de rendimiento

A 31 de julio de 2026 la red opera con normalidad (100 % de uptime en 90 días) [11]. El TVL de la cadena ronda los

\$227 M [12]. El primitivo de crédito dominante es **Blend** (\$141 M TVL, APY medio $\approx 3,6\%$): pools de préstamo aislados donde los depositantes reciben *bTokens* cuyo tipo de cambio (*b-rate*) acumula el interés por segundo, asegurados por un módulo *backstop* de primera pérdida [13]. DeFindex (\$11,5 M TVL) es una capa de vaults que *rutea depósitos hacia Blend* [14]; Upshift llegó a Soroban en junio de 2026 con el mismo modelo institucional [15]. Soroswap presenta actividad residual; Aquarius mantiene >\$40 M en AMM Soroban [12].

Tres hechos técnicos condicionan el diseño: (i) un contrato Soroban *puede* depositar directamente en Blend —lo demuestran DeFindex mismo, una implementación abierta de referencia [16] y los integradores en producción (Meru, Airtm) [17]—; (ii) el AMM nativo de Stellar classic y los *claimable balances* son **inaccesibles** desde Soroban: el Stellar Asset Contract solo expone la interfaz de token [18]; (iii) XLM no tiene staking de consenso: no existe rendimiento “nativo” de protocolo.

2.4 Gap analysis: lo que no existe en Stellar

La búsqueda sistemática (julio 2026) sobre el ecosistema, el Stellar Community Fund y los registros públicos arroja ausencias notables, que constituyen el espacio de diseño de este documento:

- G1.** ROSCA/tanda madura en producción. Solo existe SoroSusu, embrionario y sin tracción [23].
- G2.** Ahorro con metas y bloqueo temporal como producto (los primitivos existen; nadie los productizó).
- G3.** Ahorro con compromiso y penalización social (patrón GoodGhosting/HaloFi [24]).
- G4.** *Prize-linked savings* (sorteo sin pérdida, patrón PoolTogether).
- G5.** Vault de ahorro comunitario con gobernanza soulbound.
- G6.** Tooling de firmas umbral (FROST) para Stellar.
- G7.** Cliente de verificación ligera (light client) móvil.
- G8.** Malla *store-and-forward* de transacciones.
- G9.** DePIN de cualquier tipo sobre Stellar.
- G10.** Votación privada. La SDF lista “zkVoting” como caso de uso aspiracional [54]; nadie lo ha construido, y no existe port de Semaphore a Soroban [55].

Cada ausencia tiene sus *building blocks* verificados en la red. Ninguna está ensamblada. RAÍZ puede ser el primero en varias simultáneamente porque ya posee las piezas de identidad (soulbound), gobernanza y pagos de las que estos instrumentos dependen.

3 RAÍZ hoy: arquitectura desplegada

El sistema en testnet consta de cuatro contratos Soroban (Rust, soroban-sdk 22.x) y una aplicación Android nativa (Kotlin, Jetpack Compose) que opera como cliente delgado: lee por simulación vía Soroban RPC y escribe transacciones firmadas por el usuario. No existe backend propio.

- **Pool:** recibe `pay_merchant`, aplica el split (comercio / tip / fee), custodia el fondo por barrio, mantiene

el índice de comercios y hoy deposita el fondo ocioso en el vault DeFindex (`deposit_idle_to_vault / redeem_from_vault`).

- **Governance:** mintea tokens de residencia *soulbound* (sin función transfer), gestiona propuestas, voto único por residente, quórum del 30% y `tally` idempotente.
- **Treasury:** ejecución *trustless*; cualquiera invoca `execute_proposal`, el contrato verifica en Governance, rescata el yield y ordena el retiro. No custodia.
- **Rewards:** puntos no transferibles (1 punto por 0,01 USDC de tip, acuñables solo por `Pool`) y catálogo de premios de artesanos.

Los flujos de pago, gobernanza, ejecución, canje y *onboarding* están verificados extremo a extremo en dispositivo real, con 55 tests de contratos en verde. Las limitaciones conocidas —y relevantes aquí— son dos: la clave del administrador demo viaja embebida en el APK, y la acreditación de residencia es un mint manual del administrador. La Figura 1 muestra la transición arquitectónica que este documento propone; las Secciones 4–6 la desarrollan.

4 Vía 1 — Independencia de yield: Blend directo tras un adaptador propio

4.1 El cambio quirúrgico

La observación que habilita esta vía es pública y verificada: DeFindex es una capa de conveniencia cuyo subyacente es Blend [14]. RAÍZ puede, por tanto, hablar con el subyacente. El contrato `Pool` sustituye sus llamadas al vault por el flujo Blend:

- **Depósito:** `submit()` con petición `SUPPLY_COLLATERAL` sobre el pool USDC; el fondo recibe *bTokens*.
- **Valoración:** `get_reserve()` expone el *b-rate*; el valor del fondo es $v = b_{rate} \cdot q_{bTokens}$, calculable on-chain sin oráculos ni API externas. El APY se deriva de la serie temporal del propio *b-rate*.
- **Retiro:** petición de retiro simétrica; `Treasury.execute_proposal` rescata antes de pagar, como hoy.
- **Emisiones:** `claim()` recupera las emisiones BLND del pool, que pueden liquidarse a favor del fondo.

La contabilidad por barrio es idéntica a la actual (los *bTokens* sustituyen a *VaultShares*); el token custodiado no cambia (ya es el USDC compatible con Blend); la app reemplaza `DeindexClient` por un `BlendClient` de lecturas puras y elimina la API key externa. El patrón está en producción en integradores como Meru [17] y documentado en una implementación abierta de referencia [16].

4.2 YieldAdapter: de integración a protocolo

Sustituir DeFindex por Blend sin más sería cambiar una dependencia por otra. El diseño interpone una interfaz propiedad del protocolo:

<code>deposit(barrio, amount)</code>	→ <code>shares</code>
<code>withdraw(barrio, shares)</code>	→ <code>amount</code>
<code>value_of(barrio)</code>	→ <code>i128</code> (USDC)
<code>apy_hint()</code>	→ <code>u32</code> (bps, informativo)

`Pool` y los contratos de ahorro de \$5 dependen únicamente de esta interfaz. La primera implementación es `BlendAdapter`; las siguientes pueden envolver RWA (On-`do` USDY, ≈5,3% APY al lanzamiento en Stellar [20]; bonos CETES de Etherfuse [21], ambos con restricciones jurisdiccionales que el adaptador encapsula), renta fija cuando madure (*Spield*, hoy en *testnet* [22]) o carteras mixtas. El punto de diseño central: **set_adapter es una operación goberna-da**. La pregunta “¿el fondo del barrio va 100% conservador o 70/30?” se convierte en una propuesta más que los residentes votan. El ahorro también se gobierna; ningún vault del ecosistema ofrece esto.

4.3 Gestión de riesgo: la lección YieldBlox

En febrero de 2026 un pool construido *sobre* Blend (*YieldBlox*) fue drenado por ≈\$10,2M mediante manipulación de oráculo: el atacante vació la liquidez del par USTRY/USDC, infló el precio ~100× y tomó prestado contra el colateral sobrevalorado [19]. La lección operativa no es “Blend es inseguro”, sino que **el riesgo es por pool**: oráculo, activos listados y tamaño del backstop del pool concreto. Política de RAÍZ:

1. Operar únicamente contra pools con oráculos robustos y backstop profundo; parametrizar el pool objetivo en el adaptador, no en `Pool`.
2. Mantener un **colchón líquido no invertido** (p.ej. 20% del fondo) como parámetro gobernable; las ejecuciones de Treasury se sirven primero del colchón.
3. Exponer en el dashboard de transparencia el pool exacto, su backstop y su oráculo: el riesgo visible es parte de la misión.
4. A futuro, el gap G7 de \$2.4 sugiere una mutual de cobertura comunitaria; tras *YieldBlox* no existe ninguna en Stellar.

5 Vía 2 — La capa de ahorro que no existe en Stellar

Esta sección convierte a RAÍZ de “fondo que rinde” en “protocolo de ahorro”. Los cuatro instrumentos comparten infraestructura (depósitos por miembro, épocas, *YieldAdapter*, reparto) y todos se apoyan en el activo diferencial que RAÍZ ya posee: **la identidad soulbound de barrio**.

5.1 Cadena de Barrio: la ROSCA soulbound

La ROSCA (tanda, cadena, san, pasanaku, natillera) es la institución de ahorro popular dominante en América Latina: *N* personas aportan una cuota periódica y cada ronda una de ellas recibe el bote completo. Su traducción on-chain ha fracasado históricamente por el *default anónimo*: quien ya cobró

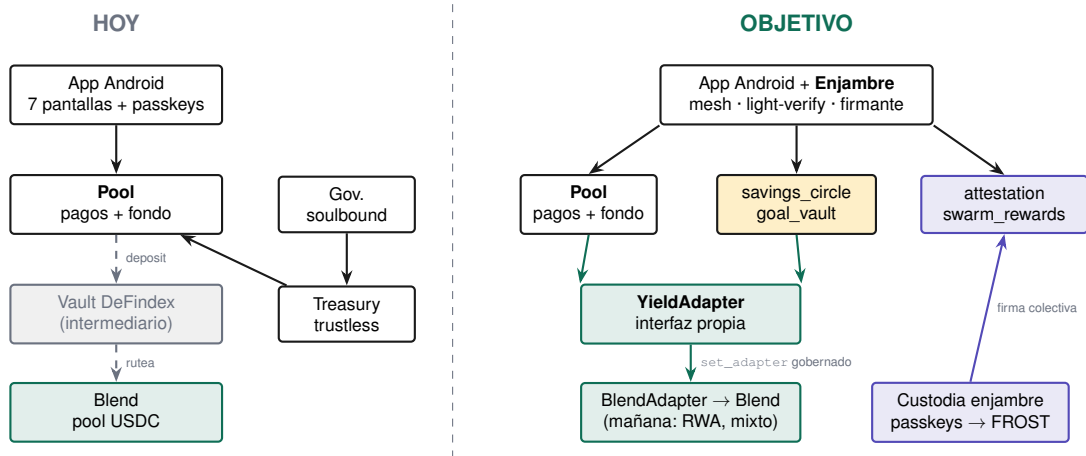


Figura 1: Transición arquitectónica. Hoy el yield pasa por un vault intermediario; en el diseño objetivo el Pool y los nuevos contratos de ahorro dependen de una interfaz propia (YieldAdapter) cuya implementación se elige por gobernanza, y la autoridad administrativa se disuelve en la custodia del enjambre.

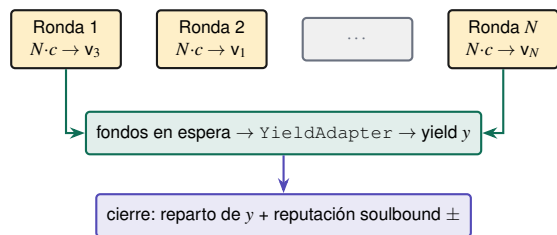


Figura 2: Ciclo de una Cadena de Barrio con N vecinos verificados (v_i), cuota c y orden de cobro sorteado σ . El bote en espera rinde; el cierre liquida el yield y y actualiza la reputación.

su turno no tiene incentivo para seguir aportando si su identidad es descartable [25, 26].

El token soulbound de Governance resuelve exactamente este problema. En la *Cadena de Barrio* (contrato `savings_circle`):

- Solo residentes verificados del barrio pueden unirse; la identidad no es descartable porque no es transferible ni re-minteable.
- El orden de cobro se asigna por sorteo (o subasta de descuento en variantes v_2); la cuota c y el período son parámetros del círculo.
- En cada ronda r , el bote $B = N \cdot c$ se transfiere al miembro $\sigma(r)$; los fondos en espera se depositan vía YieldAdapter, y el rendimiento acumulado y se liquida al cierre: a prorrata entre los cumplidores o, en círculos “solidarios”, al fondo común del barrio —la tanda le paga renta a la comunidad.
- El incumplimiento se registra on-chain: el historial de círculos completados constituye una **reputación de ahorro** pública y no falsificable, base de un futuro score crediticio comunitario (v_3) y de las políticas de admisión de círculos de mayor cuota.

En Stellar solo existe un intento embrionario de ROSCA (SoroSusu: sin pagos automáticos, sin auditoría, sin tracción [23]); ninguno con identidad fuerte. La Cadena de Ba-

rrío sería la primera ROSCA con resistencia sybil estructural del ecosistema, y el instrumento que más fielmente traduce la cultura financiera de los barrios a los que RAÍZ sirve.

5.2 Bóveda de metas con bloqueo temporal

Stellar posee *time-locks* nativos (claimable balances con predicados temporales [27]), pero son inaccesibles desde Soroban [18] y nadie los ha productizado (G2). El contrato `goal_vault` implementa metas personales o colectivas: objetivo (m, t) de monto y fecha, retiro bloqueado hasta cumplirse, depósitos de terceros admitidos —un turista puede aportar a “la escuela: 500 USDC para diciembre” visible en la app, una forma nueva de Tip dirigido— y rendimiento vía el adaptador. Sería el primer *timelock-vault* con metas de Soroban.

5.3 Retos de ahorro con compromiso

Patrón *commit-to-save* (GoodGhosting/HaloFi [24]), inexistente en Stellar (G3): temporadas de k aportes periódicos; quien cumple recupera capital más su parte del yield; el yield de quien abandona se redistribuye entre los cumplidores o cae al fondo del barrio. La deserción financia lo común.

5.4 Sorteo sin pérdida

Prize-linked savings (patrón PoolTogether; G4): el yield agregado de un período se sortea entre los ahorradores; nadie pierde capital. La aleatoriedad v_1 se deriva de datos de ledger con compromiso previo (*commit-reveal*); Protocol 25 habilita sorteos con verificación ZK a futuro [8]. El encuadre jurídico por jurisdicción (“premio al ahorro”, no lotería) debe validarse antes del despliegue.

6 Vía 3 — El Enjambre: los dispositivos del barrio como infraestructura

La intuición fundacional —“dispositivos conectados por enjambres que sirvan como nodos”— es correcta en dirección y debe afinarse en física. Un teléfono no puede ser validador de Stellar: los requisitos de hardware y disponibilidad [5] son incompatibles con móviles, NAT y batería. Y ningún sistema puede ofrecer *finalidad* de pagos offline sin hardware seguro o custodia: es la conclusión formal del programa Polaris del BIS sobre pagos digitales sin conexión [34]. Lo que sigue es lo que el enjambre *sí* puede ser —cinco roles, ninguno existente hoy en Stellar— y cómo cada rol se apoya en primitivos verificados.

6.1 El enjambre custodia: del admin embebido a la firma colectiva

La limitación más grave del MVP es la clave administrativa embebida en el APK. El enjambre la disuelve en tres etapas de descentralización creciente:

6.1.1 Etapa A — multisig nativo (disponible hoy)

La autoridad administrativa migra a una cuenta Stellar con hasta 20 firmantes-vecinos, pesos y umbrales por criticidad (*low/med/high*) [28]. Cero código nuevo de protocolo; precedente comunitario documentado (Montelibero [29]).

6.1.2 Etapa B — smart account comunal con passkeys y políticas

El fondo y la autoridad del barrio se instancian como cuenta inteligente Soroban cuyos firmantes son las **passkeys** (WebAuthn/secp256r1, CAP-51 [30]) de M residentes, con *policy signers*: umbral k -de- n , límites diarios, *timelocks* por monto, listas de operaciones permitidas. Los bloques están auditados y disponibles (OpenZeppelin Stellar Contracts, smart accounts RC v0.7.0 [31]), y la delegación nativa de Protocol 27 abarata precisamente la agrupación de firmantes delegados [10]. RAÍZ ya integra el kit de smart accounts para usuarios: extenderlo a la custodia comunal es el paso natural, y nadie lo ha ensamblado para fondos vecinales en Stellar.

6.1.3 Etapa C — FROST: el enjambre es el firmante

FROST [32] es un esquema de firmas umbral: n participantes generan colectivamente una clave de grupo mediante DKG, y cualquier subconjunto de tamaño t produce una firma. La propiedad decisiva: las implementaciones FROST-ed25519 producen **firmas ed25519 estándar** [33], indistinguibles de una firma individual, que Stellar verifica hoy **sin ningún cambio de protocolo**. La cuenta del fondo pasa a tener como clave pública la clave de grupo del enjambre: “la llave del barrio no existe; existe el barrio”. No existe tooling FROST-Stellar en ningún lugar (G6): construir *frost-stellar* (DKG entre móviles, *re-sharing* ante altas y bajas de vecinos, recuperación ante pérdida de dispositivos) es una contribución de infraestructura al ecosistema completo, del perfil que el SCF financia.

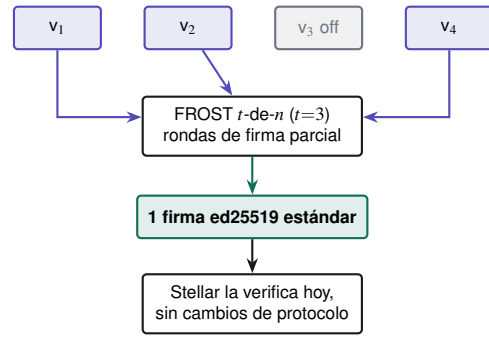


Figura 3: Custodia por enjambre en su etapa final: t de n dispositivos de residentes co-firman; la red solo ve una firma ed25519 ordinaria. (v_3 puede estar apagado: basta el umbral.)

6.2 El enjambre atestigua: residencia como red de confianza

El segundo bloqueante del MVP es el KYC simulado. Se sustituye por atestación vecinal: un aspirante es avalado por M residentes verificados que firman con su passkey en el contrato *attestation*, con depósito de reputación en juego, ventana de disputa y revocación. Es imprescindible nombrarlo con honestidad: no es una *prueba* criptográfica de presencia física —el GPS es falsificable y la co-presencia BLE prueba dispositivos, no personas [38]—, sino un mecanismo *social* de resistencia sybil con incentivos, en la tradición de las redes de confianza [39]. Para una comunidad que se conoce, es más honesto y más fiel a la misión que un KYC corporativo. No existe nada equivalente en Stellar.

6.3 El enjambre transporta: malla store-and-forward

Una transacción Stellar es un sobre XDR firmado, transportable por cualquier canal: BLE, WiFi Direct, QR, NFC. El patrón está probado fuera de Stellar: TxTenna transportó transacciones Bitcoin por radio mesh hasta un gateway con internet [35], y Bitchat (2025) demostró mensajería mesh BLE *store-and-forward* real entre smartphones [36]. En RAÍZ: el turista sin datos firma su pago; el XDR salta entre dispositivos del enjambre (comercios, vecinos) hasta que cualquiera con conectividad lo emite; la confirmación regresa por la malla. Dos límites se declaran sin eufemismos: los *sequence numbers* y *time bounds* acotan la ventana útil de una transacción pre-firmada, y el receptor **no tiene finalidad** hasta la confirmación de red —la interfaz muestra “pago en tránsito por el enjambre”—. Con ledgers de ~ 5 s, la finalidad llega segundos después de que un nodo de la malla toque internet. Resuelve un dolor real (zonas turísticas sin señal, turistas sin SIM local) y sería el primer transporte mesh de transacciones del ecosistema (G8).

6.4 El enjambre verifica: finalidad SCP en el móvil

No existe light client de Stellar (G7). El patrón para construir su versión mínima está probado por el oráculo de Stellar del puente Spacewalk (Pendulum): recolectar los sobres SCP *externalized* que difunden los validadores y

verificar las firmas ed25519 del conjunto tier-1 conocido [37]. La criptografía es trivial para un móvil. Una biblioteca *stellar-light-verify* embebida en la app hace que el dashboard de transparencia no dependa de confiar en un RPC: “*este saldo del fondo está confirmado por la red, verificado en tu teléfono*”. El supuesto de confianza (la lista tier-1, subjetividad débil) se documenta explícitamente; verifica finalidad de ledger, no inclusión arbitraria de estado.

6.5 El enjambre se recompensa: DePIN de barrio

No existe DePIN alguno sobre Stellar (G9); el sector vive en otras redes [40]. Los micropagos de Stellar (comisiones de fracción de centavo) permiten recompensar los servicios *medibles* del enjambre: relays de malla que transportaron pagos (acuses firmados), comercios-gateway que emitieron transacciones ajenas, dispositivos que ejecutan el verificador ligero, y uno a tres mini-PCs comunitarios (casa comunal, junta) que corren un nodo *watcher* como “testigo del barrio”—alimenta el dashboard e indexa los eventos locales; sin peso en el consenso global, y así se comunica—. La fuente de las recompensas cierra el círculo económico: la comisión de protocolo (0,5%) deja de acumularse en el administrador y financia la infraestructura del enjambre vía el contrato *swarm_rewards*. La lección de Helium se respeta: lo difícil no es pagar sino verificar el trabajo físico [41]; la v1 recompensa únicamente trabajos verificables on-chain.

7 Vía 4 — La capa ZK: transparencia colectiva, privacidad individual

La tesis de RAÍZ exige transparencia radical de lo colectivo: cada pago, cada ejecución del fondo, auditable por cualquiera. Pero en un barrio real donde todos se conocen, la transparencia de lo *individual* es un riesgo: el voto público expone al vecino a la presión social, y el saldo público expone al ahorrador. La capa ZK resuelve esta tensión sin traicionar la misión, bajo un principio de diseño estricto: **los agregados permanecen públicos y verificables; las atribuciones individuales se vuelven privadas**. El fondo común jamás se oculta. El momento es deliberado: la estrategia de privacidad publicada por la SDF (“transparencia por defecto, privacidad configurable, con cumplimiento”) [50] y las primitivas de Protocol 25–26 hacen de esta capa una continuación natural del protocolo, no un injerto.

7.1 Primitivas y estado del arte en Stellar (verificado julio 2026)

Protocol 25 introdujo las funciones *host BN254* (*bn254_g1_add*, *bn254_g1_mul*, *bn254_multi_pairing_check*; paridad con los precompiles EIP-196/197 de Ethereum) [43] y las permutaciones Poseidon/Poseidon2 *parametrizables* sobre los campos de BN254 y BLS12-381 [44] —lo que permite reproducir la instancia exacta de Poseidon de circomlib pasando sus constantes—. Protocol 26 añadió MSM y aritmética del

campo escalar (*bn254_g1_msm*, *bn254_fr_**) [45]. En el SDK, estas primitivas exigen *soroban-sdk* $\geq 25.x$ (recomendado 26.1) [46]; el workspace actual de RAÍZ (22.x) requiere una migración planificada.

Existen ya verificadores de referencia: el ejemplo oficial Groth16 de *soroban-examples* (sobre BLS12-381, portable a BN254 con las mismas cuatro parejas de pairing) [47], el verificador RISC Zero de Nethermind (Groth16/BN254, sin auditar) [48] y verificadores UltraHonk para Noir [49]. Los costos son el límite operativo: un pairing consume ≈ 40 M de los ~ 100 M de instrucciones por transacción [52, 53]; una verificación Groth16/BN254 completa cabe en el presupuesto pero consume gran parte de él. En el ecosistema, OpenZeppelin publicó en julio de 2026 el *developer preview* de tokens confidenciales (compromisos de Pedersen + pruebas Noir; testnet, auditorías en curso) [51], y existen prototipos de privacy pools [52]. **Votación privada no ha construido nadie** (G10): la SDF la lista como caso de uso aspiracional [54].

7.2 El problema: el voto público en un barrio real

En el contrato Governance actual, *vote()* persiste *Vote(proposal_id, Address)* y emite el evento (*vote, barrio_id*) con datos (*proposal_id, resident, support*): la identidad y el sentido del voto son doblemente públicos, en storage y en el stream de eventos. El token soulbound impide *comprar* el voto (no es transferible); no impide *coaccionarlo*: en una comunidad pequeña, votar contra la propuesta del vecino con poder tiene costo social observable. El voto secreto es un principio democrático básico que la gobernanza de RAÍZ hoy no ofrece.

7.3 Diseño: voto secreto tipo Semaphore

El diseño adopta Semaphore v4 [55] —el protocolo de señalización anónima de PSE, probado en producción masiva por World ID [57]— sin modificar su circuito:

1. **Registro.** Al acreditarse la residencia (hoy mint del admin; mañana atestación vecinal, §6.2), el teléfono genera una identidad Semaphore (clave EdDSA sobre Baby Jubjub, protegida por el mismo Keystore que ya usa la app) y su *commitment* = $\text{Poseidon}(A_x, A_y)$ se inserta en el árbol de Merkle incremental (LeanIMT) del barrio, mantenido on-chain junto al registro soulbound.
2. **Voto.** El teléfono genera una prueba Groth16/BN254 de: “conozco un secreto cuyo commitment está en el árbol de raíz *R*, y mi nullifier es $\text{Poseidon}(\text{proposal_id}, \text{secreto})$ ”. Con el tooling móvil actual (mopro, bindings Kotlin; witnessc + rapidsnark) la prueba tarda $\approx 0,17$ s en gama alta y < 1 s en gama media [56]: imperceptible en UX.
3. **Verificación.** *vote_private(proof, root, nullifier, support)*: el contrato verifica la prueba con las funciones *host BN254*, exige *root* $\in$ raíces válidas del barrio y *nullifier* inédito (anti doble voto), y suma el voto. El *tally* sigue siendo público y auditable; las identidades, no. Nótese que la firma del emisor de la transacción deja de identificar al votante:

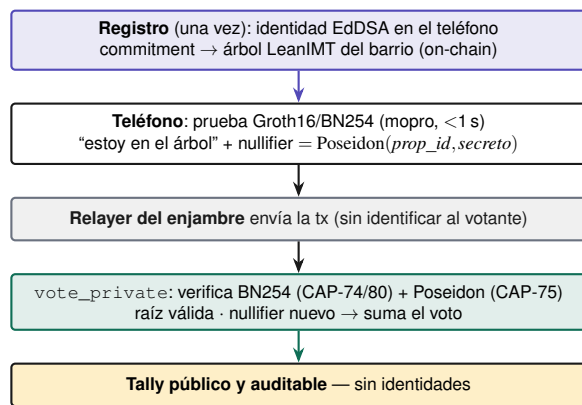


Figura 4: Voto secreto tipo Semaphore sobre Soroban. El circuito estándar v4 no se modifica (hereda su ceremonia de setup pública); el código propio se concentra en el verificador y la lógica de contrato.

cualquier *relayer* puede enviarla —y aquí la capa ZK se trenza con el Enjambre: los dispositivos del barrio actúan como relayers (§6.3), desvinculando también los metadatos de red.

Tres decisiones de ingeniería reducen el riesgo al mínimo: (a) **no escribir circuitos propios** —la clase de bug dominante en ZK son los circuitos *under-constrained* [59]; usando el circuito estándar v4 y sus *zkeys* de ceremonia pública, el código propio (y su auditoría) se concentra en el verificador Soroban y la gestión de árbol/raíces/nullifiers—; (b) validar la equivalencia Poseidon↔circomlib con vectores de prueba antes de nada; (c) el voto público actual se mantiene como modo de compatibilidad hasta que el privado esté auditado.

7.4 Fase 2: privacidad del ahorro y de la reputación

Con el mismo principio: en los instrumentos de §5, los *aportes individuales* pueden registrarse como compromisos de Pedersen cuya suma homomórfica hace público el agregado del círculo sin atribuir cifras por persona —el patrón de los tokens confidenciales de OpenZeppelin sobre Stellar [51], cuya salida de *preview* conviene esperar antes que reimplementarlo—. Una alternativa más simple y honesta para v1: denominaciones fijas de aporte (elimina los range proofs). Y para la reputación de ahorro (§5.1): probar en ZK “completé ≥ 3 círculos sin default” sin publicar el historial financiero completo —la versión digna de un buró de crédito.

7.5 Límites honestos de la capa ZK

1. **No es receipt-free:** quien conoce su secreto puede *demonstrar* cómo votó; la venta voluntaria de voto sigue siendo posible. La solución conocida (MACI [58]) exige un coordinador con infraestructura propia —desproporcionado para un barrio hoy; queda como escalada si el soborno se vuelve amenaza real.
2. **La coacción presencial es irresoluble a nivel de protocolo:** nadie impide votar con el coaccionador mirando la

pantalla. Se mitiga (ventanas largas de votación), no se elimina.

3. **El anonimato está acotado por el censo:** con 15 residentes y 12 votos, el conjunto de anonimato es pequeño. Los tallies deben publicarse agregados por cierre, no voto a voto en tiempo real.
4. **El presupuesto de cómputo es estrecho:** una verificación consume gran parte de las ~ 100 M instrucciones de una transacción; una prueba por voto, sin lotes, es el diseño v1.
5. **El tooling es joven:** los verificadores de referencia no están auditados; la migración de SDK (22.x→26.1) toca los cuatro contratos. Por eso esta vía es tercera ola (F6), nunca el critical path.

8 Modelo de amenazas y límites declarados

8.1 Lo que este diseño no promete

1. **“Los teléfonos validan Stellar”:** falso e innecesario. El enjambre custodia, atestigua, transporta y verifica; no produce consenso global [5].
2. **Finalidad offline:** imposible en software puro; el doble gasto sin conexión requiere hardware seguro o custodia [34]. La malla es transporte.
3. **“Prueba” de residencia:** la atestación vecinal es un mecanismo social con incentivos, no una prueba criptográfica de presencia [38].
4. **APY garantizado:** el rendimiento de Blend es variable y con riesgo por pool (§4.3); se muestra siempre como variable y con el riesgo visible.
5. **“Voto incoercible”:** el voto secreto ZK elimina el recibo público, no la coacción presencial ni la venta voluntaria de voto (§7.5); “anonimato” nunca se promete en censos pequeños sin cualificarlo.

8.2 Riesgos principales y mitigaciones

- **Oráculo/pool** (YieldBlox [19]): selección estricta de pool, colchón líquido, transparencia de exposición.
- **Colusión en la atestación** (sybil social): depósitos de reputación, revocación, ventanas de disputa, admisión gradual (un residente nuevo no atestigua de inmediato).
- **Pérdida de dispositivos en la custodia:** umbral $t < n$ con margen, re-sharing periódico, recuperación social vía el propio contrato de atestación.
- **Default en círculos:** cuotas iniciales bajas, reputación progresiva, variantes con colateral parcial para botes altos.
- **Regulatorio:** encuadre del sorteo sin pérdida y del off-ramp por jurisdicción; anchors SEP-24/38 para la rampa fiat en el roadmap general del proyecto.
- **Circuitos y verificadores ZK:** política de no escribir circuitos propios (Semaphore v4 estándar + ceremonia heredada [55, 59]); vectores de prueba Poseidon↔circomlib; auditoría del verificador antes de retirar el voto público de compatibilidad.

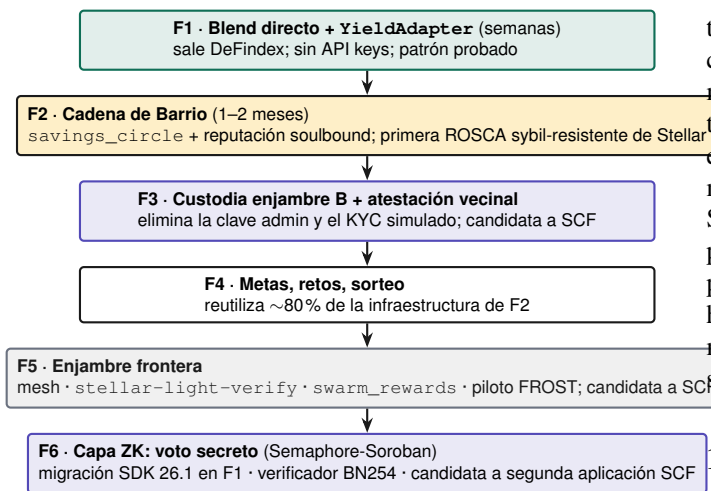


Figura 5: Fases propuestas. Cada fase es demostrable por sí sola y no interrumpe lo desplegado: el Pool actual sigue operando mientras el adaptador se valida en testnet.

9 Trabajo relacionado

En Stellar: DeFindex y Upshift ofrecen vaults de yield B2B sin gobernanza territorial ni identidad soulbound [14, 15]; SoroSusu es el único intento ROSCA, embrionario [23]; Montelibero documenta custodia multisig comunitaria clásica [29]; el oráculo de Spacewalk demuestra la verificación ligera de SCP fuera de la red [37]. **Fuera de Stellar:** WeTrust y las ROSCAs sobre Ethereum ilustran el fallo por identidad débil [26]; GoodGhosting/HaloFi el ahorro con compromiso [24]; PoolTogether el sorteo sin pérdida; TxTenna y Bitchat el transporte mesh [35, 36]; Helium la economía DePIN y sus problemas de verificación [41]; FROST (RFC 9591) la firma umbral compatible [32, 33]; y el programa Polaris del BIS fija el límite formal de los pagos offline [34]. **En privacidad:** World ID es el mayor despliegue de Semaphore en producción [57]; MACI añade resistencia al soborno a costa de un coordinador [58]; Vocdoni opera votación anónima con zk-SNARKs y el PoC de voto privado de NounsDAO (Aragon+Aztec) nunca llegó a producción —útil como frontera de lo intentado—. En Stellar, los tokens confidenciales de OpenZeppelin (preview) [51] y los prototipos de privacy pools [52] son el estado del arte; votación privada no existe. La combinación —identidad soulbound territorial + ahorro popular + custodia de enjambre + voto secreto sobre una red de pagos de coste marginal— no aparece ensamblada en ningún ecosistema.

10 Hoja de ruta

La Figura 5 ordena el trabajo por riesgo y dependencia. F1 cumple el objetivo declarado (independencia de DeFindex) con el mínimo riesgo. F2 materializa la identidad de protocolo de ahorro. F3 elimina los dos bloqueantes de mainnet con tecnología inédita en Stellar. F4 completa el portafolio de instrumentos sobre la infraestructura de F2. F5 agrupa las con-

tribuciones de infraestructura de ecosistema. F6 (capa ZK) corre como tercera ola: su único prerequisite temprano —la migración del workspace a soroban-sdk 26.1— se ejecuta durante F1, cuando el código de contratos ya está abierto; el voto secreto entra a desarrollo solo con F2 estable, y “Semaphore para Stellar” se perfila como *segunda* aplicación al SCF con vida propia. Las fases F3, F5 y F6 encajan en el perfil del Stellar Community Fund (SCF 7.0: hasta \$150 K por track de Build, tope de \$300 K por proyecto, pagos por hitos) [42], cuya cartera histórica financia precisamente “primeros en Stellar” de infraestructura —y la propia SDF ha señalado la votación privada como caso de uso deseado [54].

11 Conclusión

RAÍZ demostró en testnet que una economía de barrio puede operar on-chain: pagos, fondo común, gobernanza soulbound y ejecución sin custodios. La dependencia de DeFindex es eliminable con riesgo mínimo porque su subyacente —Blend— es directamente accesible desde Soroban, y el adaptador propio convierte la fuente de rendimiento en una decisión gobernada por los residentes. La diferenciación no está en competir con los vaults existentes sino en construir la capa que el ecosistema no tiene: los instrumentos de ahorro popular latinoamericanos con identidad soulbound, y un enjambre de dispositivos vecinales que custodia, atestigua, transporta, verifica y se recompensa. Cada componente descansa en primitivos verificados de la red a julio de 2026 —incluida la delegación de autenticación recién activada en Protocol 27 y las primitivas ZK de Protocol 25–26— y ninguno existe ensamblado. La capa ZK, en particular, convierte la última tensión del diseño —transparencia colectiva frente a intimidad individual— en una elección de arquitectura en lugar de un compromiso. La misión no cambia; se profundiza: el barrio deja de ser beneficiario de la infraestructura y pasa a serla.

Referencias

- [1] D. Mazières. *The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus*. Stellar Development Foundation. <https://stellar.org/papers/stellar-consensus-protocol>
- [2] SDF. *Stellar Consensus Protocol — Fundamentals*. <https://developers.stellar.org/docs/learn/fundamentals/stellar-consensus-protocol>
- [3] SDF. *Tier 1 Organizations*. <https://developers.stellar.org/docs/validators/tier-1-orgs>
- [4] TechAfrica News. *MoneyGram, Figure Markets and Range join Stellar as Tier-1 validators* (16-jul-2026). <https://techafricanews.com/2026/07/16/>
- [5] SDF. *Validator Admin Guide — Prerequisites*. <https://developers.stellar.org/docs/validators/admin-guide/prerequisites>
- [6] SDF. *Announcing Protocol 23*. <https://stellar.org/blog/developers/announcing-protocol-23>
- [7] SDF. *Protocol 24 Upgrade Guide*. <https://stellar.org/blog/developers/protocol-24-upgrade-guide>

- [8] SDF. *Announcing Stellar X-Ray: Protocol 25*. <https://stellar.org/blog/developers/announcing-stellar-x-ray-protocol-25>
- [9] SDF. *Stellar Yardstick: Protocol 26 Upgrade Guide*. <https://stellar.org/blog/foundation-news/stellar-yardstick-protocol-26-upgrade-guide>
- [10] SDF. *Stellar Zipper: Protocol 27 Upgrade Guide*. <https://stellar.org/blog/foundation-news/stellar-zipper-protocol-27-upgrade-guide>
- [11] SDF. *Stellar Status*. <https://status.stellar.org>
- [12] DefiLlama. *Stellar Chain Overview* (consultado 31-jul-2026). <https://defillama.com/chain/stellar>
- [13] Blend Capital. *Blend Whitepaper*. <https://docs.blend.capital/blend-whitepaper>
- [14] SDF. *Composability on Stellar: From Concept to Reality*. <https://stellar.org/blog/developers/composability-on-stellar-from-concept-to-reality>
- [15] LumenLoop. *Upshift brings multi-chain vault infrastructure to Stellar* (jun-2026). <https://lumenloop.com/research/>
- [16] J. Bachini. *Blend Vault: auto-compounding Soroban vault* (código de referencia). <https://github.com/jamesbachini/Blend-Vault>
- [17] SDF. *Meru wallet uses Blend DeFi protocol for yield* (case study). <https://stellar.org/case-studies/meru-wallet-uses-blend-defi-protocol-for-yield>
- [18] SDF. *Stellar Asset Contract*. <https://developers.stellar.org/docs/tokens/stellar-asset-contract>
- [19] Halborn. *Explained: The YieldBlox Hack* (February 2026). <https://www.halborn.com/blog/post/explained-the-yieldblox-hack-february-2026>
- [20] Ondo Finance / SDF. *Ondo launches USDY on Stellar*. <https://stellar.org/press/ondo-finance-launches-usdy-on-stellar>
- [21] SDF. *Global Bonds, Local Impact: Stablebonds on Stellar*. <https://stellar.org/blog/foundation-news/global-bonds-local-impact-stablebonds-on-stellar>
- [22] Spield. *Fixed yield on Stellar* (testnet). <https://www.spield.live/>
- [23] SoroSusu Protocol. *sorosusu-contracts*. <https://github.com/SoroSusu-Protocol/sorosusu-contracts>
- [24] GoodGhosting/HaloFi. *Commit-to-save pools* (código). <https://github.com/good-Ghosting/>
- [25] *Rotating savings and credit association*. https://en.wikipedia.org/wiki/Rotating_savings_and_credit_association
- [26] CoinIdol. *WeTrust launches Trusted Lending Circles (ROSCA) platform*. <https://coinidol.com/wetrust-launches-trusted-lending-rosca-platform/>
- [27] Stellar Protocol. *CAP-0023: Two-Part Payments (Claimable Balances)*. <https://github.com/stellar/stellar-protocol/blob/master/core/cap-0023.md>
- [28] SDF. *Signatures and Multisig*. <https://developers.stellar.org/docs/learn/fundamentals/transactions/signatures-multisig>
- [29] Montelibero. *stellar-multisig* (tooling comunitario). <https://github.com/montelibero-org/stellar-multisig>
- [30] SDF. *Announcing Protocol 21 (CAP-51: secp256r1)*. <https://stellar.org/blog/developers/announcing-protocol-21>
- [31] OpenZeppelin. *Stellar Contracts: Smart Accounts* (auditado, RC v0.7.0). <https://docs.openzeppelin.com/stellar-contracts/accounts/smart-account>
- [32] C. Komlo, I. Goldberg et al. *RFC 9591: The FROST Flexible Round-Optimized Schnorr Threshold Signature Scheme*. <https://www.rfc-editor.org/rfc/rfc9591.html>
- [33] Taurus. *frost-ed25519: firmas umbral compatibles con verificación ed25519 estándar*. <https://github.com/taurushq-io/frost-ed25519>
- [34] BIS Innovation Hub. *Project Polaris: handbook for offline payments with CBDC*. <https://www.bis.org/publ/othp64.htm>
- [35] Coin Center. *Decentralizing Bitcoin's last mile with mobile mesh networks*. <https://coincenter.org/decentralizing-bitcoins-last-mile-with-mobile-mesh-networks/>
- [36] CNBC. *Bitchat: Bluetooth mesh store-and-forward messaging* (jul-2025). <https://www.cnbc.com/2025/07/07/jack-dorsey-whatsapp-bluetooth.html>
- [37] Pendulum. *Introducing the Stellar Oracle* (Spacewalk). <https://medium.com/pendulum-chain/introducing-the-stellar-oracle-ce4b85244cc8>
- [38] XYO / FOAM (prior art de proof-of-location y sus límites). <https://xyo.network/proof-of-location>
- [39] Frontiers in Blockchain. *Sybil-resistant identity and web-of-trust mechanisms (survey)*. <https://www.frontiersin.org/journals/blockchain/articles/10.3389/fbloc.2020.590171/full>
- [40] arXiv. *A Survey of Decentralized Physical Infrastructure Networks (DePIN)*. <https://arxiv.org/html/2406.02239v1>
- [41] Messari. *Understanding Helium: A Comprehensive Overview*. <https://messari.io/report/understanding-helium-a-comprehensive-overview>
- [42] SDF. *Stellar Community Fund*. <https://communityfund.stellar.org>
- [43] Stellar Protocol. *CAP-0074: Host functions for BN254*. <https://github.com/stellar/stellar-protocol/blob/master/core/cap-0074.md>
- [44] Stellar Protocol. *CAP-0075: Poseidon/Poseidon2 permutations*. <https://github.com/stellar/stellar-protocol/blob/master/core/cap-0075.md>
- [45] Stellar Protocol. *CAP-0080: Host functions for efficient ZK BN254 use cases*. <https://github.com/stellar/stellar-protocol/blob/master/core/cap-0080.md>
- [46] SDF. *rs-soroban-sdk releases* (v25.0.0: BN254 + Poseidon vía CryptoHazmat; v26.x: CAP-0080). <https://github.com/stellar/rs-soroban-sdk/releases>
- [47] SDF. *soroban-examples: groth16_verifier*. https://github.com/stellar/soroban-examples/tree/main/groth16_verifier
- [48] Nethermind. *stellar-risc0-verifier* (Groth16/BN254; no auditado). <https://github.com/NethermindEth/stellar-risc0-verifier>
- [49] indextree. *ultrahonk_soroban_contract* (verificador Noir/UltraHonk; enlazado desde los docs ZK de Stellar). https://github.com/indextree/ultrahonk_soroban_contract
- [50] SDF. *A Strategy for Privacy on Blockchain*. <https://stellar.org/blog/ecosystem/strategy-for-privacy-on-blockchain>
- [51] SDF/OpenZeppelin. *Developer Preview: Confidential Tokens on Stellar* (jul-2026). <https://stellar.org/blog/developers/developer-preview-confidential-tokens-on-stellar>
- [52] SDF. *Prototyping Privacy Pools on Stellar* (pairing $\approx 40M$ instrucciones). <https://stellar.org/blog/ecosystem/prototyping-privacy-pools-on-stellar>
- [53] noir-lang. *Discusión 8509: costes de verificación en Soroban* (límite $\sim 100M$ instrucciones). <https://github.com/orgs/noir-lang/discussions/8509>

- [54] SDF. *5 Real-World Zero-Knowledge Use Cases* (zkVoting como caso aspiracional). <https://stellar.org/blog/developers/5-real-world-zero-knowledge-use-cases>
- [55] PSE. *Semaphore Protocol v4* (docs y circuitos). <https://docs.semaphore.pse.dev/>
- [56] PSE. *mopro: mobile proving* (bindings Kotlin; benchmarks Semaphore-32 $\approx 0,17s$ en gama alta). <https://zkmopro.org/docs/performance>
- [57] World. *Intro to ZKPs, Semaphore and World ID*. <https://world.org/blog/world/intro-zero-knowledge-proofs-semaphore-application-world-id>
- [58] PSE. *MACI: Minimal Anti-Collusion Infrastructure*. <https://maci.pse.dev/docs/introduction>
- [59] 0xPARC. *ZK Bug Tracker* (circuitos under-constrained como clase dominante). <https://github.com/0xPARC/zk-bug-tracker>